Servant

Documentation
Home: https://docs.servant.dev/en/stable/index.html
Servant.Server: https://hackage.haskell.org/package/servant-server-0.19.2
Servant: https://hackage.haskell.org/package/servant-0.19.1
Servant.Client: https://hackage.haskell.org/package/servant-client-0.19

Servant.Server

```
--
module Servant.Server
serve :: HasServer api '[] => Proxy api -> Server api ->
Application
serveWithContext ::
  (HasServer api context, ServerContext context)
  => Proxy api
  -> Context context
  -> Server api
  -> Application
serveWithContextT ::
  HasServer api context
  => Proxy api
  -> Context context
  -> (forall x. m x -> Handler x)
  -> ServerT api m
  -> Application
type ServerContext context = HasContextEntry (context .+
+ DefaultErrorFormatters) ErrorFormatters


-- Server
class HasServer api context where
 type ServerT api (m :: Type -> Type) :: Type
                 -- m ~ Handler
 route ::
   Proxy api
   -> Context context
   -> Delayed env (Server api)
   -> Router env
 hoistServerWithContext ::
  Proxy api
  -> Proxy context
  -> (forall x. m x -> n x)
  -> ServerT api m
  -> ServerT api n

type Server api = ServerT api Handler
data EmptyServer

emptyServer m :: ServerT EmptyAPI m
layout :: HasServer api '[] => Proxy api -> Text
layoutWithContext :: HasServer api context => Proxy api
-> Context context -> Text
```

```
hoistServer :: HasServer api '[] => (forall x. m x -> n
x) -> ServerT api m -> ServerT api n


-- Handler
newtype Handler a = Handler {runHandler' :: ExceptionT
ServerError IO a}
instance Monad Handler, MonadIO Handler, MonadFail
Handler,
  MonadError ServerError Handler,
  MonadMask Handler, MonadCatch Handler, MonadThrow
Handler,
  MonadBase IO Handler, MonadBaseControl IO Handler


-- Errors
data ServerError = ServerError {
  errHttpCode :: Int,
  errResponsePhrase :: String,
  errBody :: ByteString,         -- Data.ByteString.Lazy
  errHeaders :: [Header]
 }
err400 :: ServerError            -- 400 Bad Request
err401 :: ServerError            -- 401 Unauthorized
err402 :: ServerError            -- 402 Payment Required
err403 :: ServerError            -- 403 Forbidden
err404 :: ServerError            -- 404 Not Found
...
```

HasServer instances

```
instance HasServer EmptyAPI context           -- ServerT
EmptyAPI m = Tagged m EmptyServer
instance HasServer Raw context                -- ServerT
Raw m = Tagged m Application
instance (HasServer a context, HasServer b
context) => HasServer (a :<|> b) context
                -- ServerT (a :<|> b) m = ServerT a
m :<|> ServerT b m
instance (KnownSymbol path, HasServer api context) =>
HasServer (path :> api) context
       -- ServerT (path :> api) m = ServerT api m

instance (KnownSymbol capture, HasServer api
context, ... ) => HasServer (Capture' mods capture a :>
api) context      -- ServerT (Capture' mods capture a :>
api) m = a -> ServerT api m
instance (KnownSymbol capture, HasServer api
context, ... ) => HasServer (CaptureAll capture a :>
api) context         -- ServerT (CaptureAll capture a :>
api) m = [a] -> ServerT api m
instance HasServer api context => HasServer (Description
d :> api) context
instance HasServer api context => HasServer (Summary
```

```
d :> api) context
instance (KnownSymbol sym, HasServer api context, ...)
=> HasServer (QueryParam' mods sym a :> api) context
         -- ServerT (QueryParam' mods sym a :> api) m =
a -> ServerT api m
instance (KnownSymbol sym, HasServer api context, ...)
=> HasServer (QueryParams sym a :> api) context
        -- ServerT (QueryParams sym a :> api) m = [a] ->
ServerT api m
instance (HasServer api context, ...) => HasServer
(ReqBody' mods list a :> api) context
           -- ServerT (ReqBody' mods list a :> api) m =
a -> ServerT api m
instance (HasServer api context, ...) => HasServer (Verb
method status ctypes a :> api) context
     -- ServerT (Verb method status ctypes a) m = m a
instance (HasServer api context, ...) => HasServer (Verb
method status ctypes (Headers h a) :> api)
context            -- ServerT (Verb method status ctypes
(Headers h a)) m = m (Headers h a)
```

Context

```
-- module Servant.Server
data Context context where
 EmptyContext :: Context '[]
 (:.) :: x -> Context xs -> Context (x ': xs)
             -- context as list of types

(.++) :: Context l1 -> Context l2 -> Context (l1 .++
l2)          -- concat (ctx1 .++ ctx2)
type family (l1 :: [Type]) .++ (l2 :: [Type]) where

  '[] .++ a = a
  (a ': as) .++ b = a ': (as .++ b)

class HasContextEntry (context :: [Type]) (val :: Type)
where
 getContextEntry :: Context context -> val

instance HasContextEntry (val ': xs) val
             -- get value if first in list
instance HasContextEntry xs val =>
             -- recursively search if not first
  HasContextEntry (notIt ': xs) val
```

ErrorFormatter

```
-- module Servant.Server
type ErrorFormatter = TypeRep -> Request -> String ->
ServerError
```

```
type NotFoundErrorFormatter = Request -> ServerError
defaultErrorFormatters :: ErrorFormatters

data ErrorFormatters
bodyParseErrorFormatter :: ErrorFormatters ->
ErrorFormatter
urlParseErrorFormatter :: ErrorFormatters ->
ErrorFormatter
headerParseErrorFormatter :: ErrorFormatters ->
ErrorFormatter
notFoundErrorFormatter :: ErrorFormatters ->
NotFoundErrorFormatter

type DefaultErrorFormatters = '[ErrorFormatters]
                     -- default
```

Servant API
Servant: https://hackage.haskell.org/package/servant

API

```
-- module Servant.API
data path :> a
    -- [infix 3] "a" :> "b" :> "c" = /a/b/c
data a :<|> b = a :<|> b
    -- [infix 4] "a" :> "b" :<|> "c"    = a/b/ or c

    --           "a" :> ("b" :<|> "c")  = a/b/ or a/c

-- module Servant.API.Modifiers
data Required
data Optional
data Strict                                        -- Strict
evaluation
data Lenient                                       --
Evaluate, or original text if it fails i.e. Either Text
a

-- module Servant.API.ContentTypes
data JSON
data PlainText
data FormUrlEncoded
data OctetStream
class Accept ctype where
 contentType :: Proxy ctype -> MediaType
 contentTypes :: Proxy ctype -> NonEmpty MediaType
class Accept ctype => MimeRender ctype a where
 mimeRender :: Proxy ctype -> a -> ByteString
class Accept ctype => MimeUnrender ctype a where
 mimeUnrender :: Proxy ctype -> ByteString -> Either
String a
 mimeUnrenderWithType :: Proxy ctype -> MediaType ->
ByteString -> Either String a
```

```
-- module Servant.API.Header
type Header = Header' [Optional, Strict]
data Header' (mods :: [Type]) (sym :: Symbol) a


--
module Servant.API.QueryParam
data QueryFlag (sym :: Symbol)
type QueryParam = QueryParam' [Optional, Strict]
data QueryParam' (mods :: [Type]) (sym :: Symbol) a
data QueryParams (sym :: Symbol) a


--
module Servant.API.Capture
type Capture = Capture' '[]
data Capture' (mods :: [Type]) (sym :: Symbol) a
data CaptureAll (sym :: Symbol) a


--
module Servant.API.ReqBody
type ReqBody = ReqBody' '[Restricted, Strict]
data ReqBody' (mods :: [Type]) (contentTypes :: [Type])
a


--
module Servant.API.Verbs
data StdMethod = GET | POST | HEAD | PUT | DELETE |
TRACE | CONNECT | OPTIONS | PATCH
data Verb (k :: StdMethod) (statusCode :: Nat)
(contentTypes :: [Type]) (a :: Type)
class ReflectMethod a where
  reflectMethod :: Proxy a -> Method

type Get = Verb 'GET 200
type Post = Verb 'POST 200
type Put = Verb 'PUT 200
type Patch = Verb 'PATCH 200
type Delete = Verb 'DELETE 200
type PostCreated = Verb 'POST 201
type PutCreated = Verb 'PUT 201
type GetAccepted = Verb 'GET 202
...
```

API as type
API specs are specified with a purely **type level DSL**.
API is implemented by associating a **Handler** with the API

type MyAPI = API as type

```
    "api" :>
    "name" :> (
     Get '[JSON] [String]                                e.g. GET /api/name
    -> [String] response
     :> Capture "uid" :> Get '[JSON] String                    GET /api/
name/:uid      -> String response
  )

  myHandler :: ServerT MyAPI Handler                  Handler
  myHandler =
     return ["ABC"]
     :<|> (\uid -> return "ABC"
```

Basic

```
----  ServerT instances i.e. handlers
--
-- Ref: https://hackage.haskell.org/package/servant-
server-0.19.2/docs/Servant-Server.html#g:3
--
handler :: Server (Get '[JSON] String)
handler = do                                          --
Handler i.e. ExceptionT ServerError IO a
  liftIO $ putStrLn "Hello"
  return "Hello"
handler :: Server (Put '[JSON] ())
handler = return ()


-- GET /user/
-- GET /user/name
-- GET /info
type API = "user :> (
   Get '[JSON] String
   :<|> "name" :> Get '[JSON] String)
 :<|> "info" :> Get '[JSON] String
handler :: Server API
handler = ((do        -- GET /user            Handler
String
  return "")
 :<|> (do             -- GET /user/name       Handler
String
  return ""))
 :<|> (do             -- Get /Info            Handler
String
  return "")



-- GET /user/data
handler :: Server ("user" :> "data" :> Get '[JSON]
String)
handler :: Handler String
                          -- same
handler = return "Hello"
```

```
-- GET /user/:uid
handler :: Server ("user" :> Capture "uid" String :>
Get '[JSON] String)
handler :: String -> Handler String
                            -- same
handler uid = return ""


-- GET /user&name=Yui
handler :: Server ("user" :> QueryParam' '[Required]
"name" String :> Get '[JSON] String)
handler :: String -> Handler String
                            -- same
handler name = return ""

-- GET /user&names[]=a&names[]=b
handler :: Server ("user" :> QueryParams "names"
String :> Get '[JSON] String)
handler :: [String] -> Handler String
                           -- same
handler names = ...

-- GET /user&test
handler :: Server ("user" :> QueryFlag "test" :>
Get '[JSON] String)
handler :: Bool -> Handler String
                           -- same
handler testEnabled =  return ""


-- POST /user
-- Request body: {name: "Yui"}
data User = User {name :: String} deriving (Generics,
FromJSON, ToJSON)
handler :: Server ("user" :> ReqBody '[JSON] User :>
Post '[JSON] ())
handler user = do
  ...
  return ()


-- e.g. combined
--
-- GET /user/:uid
-- POST /user/:uid
-- DELETE /user/:uid
type API =  "user" :> Capture "uid" String :>
 (Get '[JSON] String
 :<|> ReqBody '[JSON] User :> PostCreated '[JSON] ()
 :<|> Delete '[JSON] ())
handler :: Server API
handler uid = getUser uid          -- GET handler
 :<|> createUser uid               -- POST handler
 :<|> deleteUser uid               -- DELETE handler

getUser :: String -> Handler String
```

```
getUser uid = return ""
createUser :: String -> User -> Handler ()
createUser uid user = do
  ...
  return ()
deleteUser :: String -> Handler ()
deleteUser uid = do
  ...
  return ()


---- Response options
--
-- response formats
handler :: Server ("data" :> Get '[JSON, Plaintext]
String)
handler = return "Hello"
```

```
GET /data
Accept: application/json                          Ok.
JSON

GET /data
Accept: text/plain; charset=utf8                  Ok.
Plaintext
```

```
-- response headers
handler :: Server (
  "data" :> Get '[JSON] (Headers '[Header "A" Int,
Header "B" Bool] String)
handler = addHeader 1 (addHeader True "Hello")
```

```
HTTP/1.1 200 OK
A: 1
B: true

Hello
```

```
---- Error reporting
--
-- Use
--
--    ExceptT ServerError
--
{-# LANGUAGE OverLoadedString #- }
import Control.Monad.Except
err400 :: ServerError
throwError err400                              -- 400
Error
throwError err404
```

```
throwM err400                                   -- BAD.
Don't throw as MonadCatch. Will not be caught
                                                -- i.e.
becomes 500 internal server error
throwError (userError "")                       -- ERROR.
Must be a ServerError

throwError $ ServerError {                       -- custom
error
  errHttpCode = 400,
  errResponsePhrase = "BAD",
  errBody = "OOPS",
  errHeaders = []
}
throwError err400 { errBody = "OOPS" }          -- override
```

==========

Internals

Servant.Server.Internal.Delayed

```
data Delayed env c where
  Delayed :: {
    capturesD :: env -> DelayedIO captures
  , methodD :: DelayedIO ()
  , authD :: DelayedIO auth
  , acceptD :: DelayedIO ()
  , contentD :: DelayedIO contentType
  , paramsD :: DelayedIO params
  , headersD :: DelayedIO headers
  , bodyD :: contentType -> DelayedIO body
  , serverD :: captures -> params -> headers -> auth ->
body -> Request -> RouteResult c
 } -> Delayed env c


emptyDelayed :: RouteResult a -> Delayed env a
addCapture :: Delayed env (a -> b) -> (captured ->
DelayedIO a) -> Delayed (captured, env) b
addParameterCheck :: Delayed env (a -> b) -> DelayedIO a
-> Delayed env b
addHeaderCheck :: Delayed env (a -> b) -> DelayedIO a ->
Delayed env b
addMethodCheck :: Delayed env a -> DelayedIO () ->
Delayed env a
addAuthCheck :: Delayed env (a -> b) -> DelayedIO a ->
Delayed env b
addBodyCheck :: Delayed env (a -> b) -> DelayedIO c ->
(c -> DelayedIO a) -> Delayed env b
addAcceptCheck :: Delayed env a -> DelayedIO () ->
Delayed env a
passToServer :: Delayed env (a -> b) -> (Request -> a)
```

```
-> Delayed env b
runDelayed :: Delayed env a -> env -> Request ->
ResourceT IO (RouteResult a)
runAction ::
  Delayed env (Handler a)
  -> env
  -> Request
  -> (RouteResult Response -> IO r)
  -> (a -> RouteResult Response)
  -> IO r
```

Servant.Server.Internal.DelayedIO

```
newtype DelayedIO a = DelayedIO {
  runDelayedIO' :: ReaderT Request (ResourceT
(RouteResultT IO)) a
}
instance MonadIO DelayedIO, MonadResource DelayedIO,
MonadThrow DelayedIO,
  MonadBaseControl IO DelayedIO

liftRouteResult :: RouteResult a -> DelayedIO a
runDelayedIO :: DelayedIO a -> Request -> ResourceT IO
(RouteResult a)
delayedFail :: ServerError -> DelayedIO a
delayedFailFatal :: ServerError -> DelayedIO a
withRequest :: (Request -> DelayedIO a) -> DelayedIO a
```

Servant.Server.Internal.Router

```
type Router env = Router' env RoutingApplication

data Router' env a =
  StaticRouter (Map Text (Router' env a)) [env -> a]
  | CaptureRouter [CaptureHint] (Router' (Text, env) a)
  | CaptureAllRouter [CaptureHint] (Router' ([Text],
env) a)
  | RawRouter (env -> a)
  | Choice (Router' env a) (Router' env a)

pathRouter :: Text -> Router' env a -> Router' env a
leafRouter :: (env -> a) -> Router' env a
choice :: Router' env a -> Router' env a -> Router' env
a

data RouterStructure =
  StaticRouterStructure (Map Text RouterStructure) Int
  | CaptureRouterStructure [CaptureHint] RouterStructure
  | RawRouterStructure
  | ChoiceStructure RouterStructure RouterStructure

routerStructure :: Router' env a -> RouterStructure
```

```
sameStructure :: Router' env a -> Router' env b -> Bool
routerLayout :: Router' env a -> Text
tweakResponse :: (RouteResult Response -> RouteResult
Response) -> Router env -> Router env
runRouter :: NotFoundErrorFormatter -> Router () ->
RoutingApplication
runRouterEnv :: NotFoundErrorFormatter -> Router env ->
env -> RoutingApplication
runChoice :: NotFoundErrorFormatter -> [env ->
RoutingApplication] -> env -> RoutingApplication
worseHTTPCode :: Int -> Int -> Bool

data CaptureHint = CaptureHint {
    captureName :: Text
    captureType :: TypeRep
}
toCaptureTag :: CaptureHint -> Text
toCaptureTags :: [CaptureHint] -> Text
```

Servant.Server.Internal.RouteResult

```
data RouteResult a
  = Fail ServerError
  | FailFatal !ServerError
  | Route !a

newtype RouteResultT m a = RouteResultT {
    runRouteResultT :: m (RouteResult a)
}
```

Servant.Server.Internal.RoutingApplication

```
type RoutingApplication = Request
         -- Network.Wai.Request, Response,
ResponseReceived
  -> (RouteResult Response -> IO ResponseReceived)
  -> IO ResponseReceived

toApplication :: RoutingApplication -> Application
```

Context (e.g. HasServer api context) vs **env** (Router env)

Context	**server** settings (e.g. error handlers) used on **server start**
env	**request** args (e.g. URL captures) provided **per-request**.

e.g.

```
 serveWithContext ::
   (HasServer api, ServerContext context)
   => Proxy api
```

```
    -> Context context                                    Start
server with given Context
    -> Server api
    -> Application

  data Router' env =
    CaptureRouter [CaptureHint] (Router' (Text, env) a)   Add
new Text capture to env
    | ...
```

Server, ServerT
Handler
Router
An API server is represented as a **chain of processes** on the HTTP request.

HasServer a type of process in the chain
ServerT + base IO layer **Handler** high level **interface** of the process used by API developers
Router + base IO layer **DelayedIO** low level **implementation** of the process i.e. read **HTTP request**, compute **HTTP response**

e.g.

```
  HasServer ("a" :> api)
read path in request path
  type ServerT ("a" :> api) Handler = ServerT api Handler


  HasSerrver (QueryParam' '[Required] "name" String :> api)
parse request query param          e.g. ?name=abc
  type ServerT (QueryParam' '[Required] "name" String :> api) Handler
       = String -> ServerT api Handler

  HasServer Verb "GET' 200 '[JSON] String
return a String value
  type ServerT (Verb "GET' 200 '[JSON] String) Handler
       = Handler String
```

Types

```
  newtype Handler =                                       Base
IO layer for a Server
    Handler { runHandler' :: Except ServerError IO a }

  type Server api = ServerT api Handler

  class HasServer api context where                       Define
a server i.e. a HTTP request handler
    type ServerT api (m :: Type -> Type) :: Type
ServerT = Type of the handler
    route :: Proxy api                                    Run
handler
       -> Context context
       -> Delayed env (Server api)
       -> Router env


  newtype DelayedIO a = DelayedIO {                       Base
IO layer when routing
    runDelayedIO' :: ReaderT Request
     (ResourceT (RouteResultT IO) a
```

```
  type Router env = Router' env RoutingApplication
  data Router' env a =                                                env =
request params (e.g. captured values) passed to handler
    StaticRouter
StaticRouter = either route path OR execute handler
       (Map Text (Router' env a))                                     paths
to route.
       [env -> a]
handlers to execute.
     | CaptureRouter
CaptureRouter = add a URL capture to env   CaptureHint
       [CaptureHint]                                                  name
of the capture e.g. used by the layout function
       (Router' (Text, env) a)
     | CaptureAllRouter [CaptureHint] (Router' ([Text], env) a)
     | RawRouter (env -> a)
     | Choice (Router' env a) (Router' env a)

  data Delayed env (Server api) where
    Delayed :: {
      capturesD :: env -> DelayedIO captures
    , methodD :: DelayedIO ()
    , authD :: DelayedIO auth
    , acceptD :: DelayedIO ()
    , contentD :: DelayedIO contentType
    , paramsD :: DelayedIO params
    , headersD :: DelayedIO headers
    , bodyD :: contentType -> DelayedIO body
    , serverD :: captures -> params -> headers
       -> auth -> body -> Request -> RouteResult (Server api)
    } -> Delayed env (Server api)

  type RoutingApplication = Request
RoutingApplication = Wrapper of Wai's Application = Request ->
(Response -> IO ResponseReceived) -> IO ResponseReceived
                                    ResponseReceived = proof that
response was sent
    -> (RouteResult Response -> IO ResponseReceived)
    -> IO ResponseReceived
```